Medallion ETL Pipeline

Sales & Forecast Analytics · Galaxy Schema · Power BI Ready

Version	1.0
Schema Type	Galaxy (Fact Constellation)
Data Volume	~298K sales · 33 forecast rows
Date Coverage	2008-01-01 → 2009-12-31
BI Target	Power BI / Any BI Tool

1. Project Overview

This project implements a three-layer Medallion ETL architecture (Bronze → Silver → Gold) to transform raw retail sales and forecast JSON files into a clean, validated Galaxy Schema dataset ready for Power BI analytics.

1.1 Data Sources

Source File	Format	Content	Volume
Sales.json	JSON	Transactional retail sales	~298,246 records
forecast.json	JSON	2009 brand-country forecasts	33 records

1.2 Output Files

Output File	Type	Rows	Description
Dim_Brand.csv	Dimension	11	Brand lookup table
Dim_Country.csv	Dimension	3	Country reference
Dim_Customer.csv	Dimension	8,868	Customer profiles
Dim_Product.csv	Dimension	2,495	Product catalog
Dim_City.csv	Dimension	306	City/geography
Dim_Date.csv	Dimension	731	Calendar date spine
Fact_Sales.csv	Fact	298,246	Sales transactions
Fact_Forecast_2009.csv	Fact	33	2009 forecast targets

2. ETL Logic — Medallion Architecture

2.1 Bronze Layer — Raw Ingestion

The Bronze layer ingests both JSON source files as-is into Pandas DataFrames with no transformation. Its purpose is purely observational: understand the shape of raw data before any changes are made.

Step	Description
Load Sales.json	Read JSON array into DataFrame; capture load time and row count
Load forecast.json	Read JSON array into DataFrame
Schema Inspection	Print dtypes, column names, and .info() summary for both datasets
Volume Metrics	Count unique products, customers, cities, countries, brands, dates
Cardinality Analysis	Report unique value counts per key column
Null Census	Count and percentage of nulls per column, sorted by severity

2.2 Silver Layer — Cleansing & Standardization

The Silver layer applies all cleaning and standardization transformations to produce two clean DataFrames: silver_sales and silver_forecast.

Text Cleaning

- All string columns have leading/trailing whitespace stripped via .str.strip()
- Customer Name nulls replaced with 'Customer_<key>' surrogate identifiers
- Education and Occupation nulls filled with 'Unknown'

Date Processing

- OrderDate (string MM/DD/YYYY) parsed to datetime via pd.to_datetime with format='%m/%d/%Y'
- DateKey generated as integer offset from anchor date 2008-01-01: DateKey = (date - anchor).days + 1
- Forecast year 2009 resolved to 2009-01-01; ForecastDateKey = 367 (day 367 from anchor)

Type Casting

- ProductKey, CustomerKey, Quantity, OrderDateKey cast to int32
- Net Price cast to float64
- Forecast value cast to int64; ForecastDateKey to int32

Outlier Detection (IQR)

- IQR fences calculated for Quantity and Net Price
- Outlier count reported for observability — no rows removed (outlier flagging only)

Color Anomaly Detection

- Color field in raw sales coincidentally equals Subcategory for some records

- A Color_DQ_Flag column marks such rows as 'SUSPECT_EQUALS_SUBCATEGORY' vs 'OK'
- In Dim_Product, color is re-derived by extracting the last word of Product Name and matching against a 14-color palette; unmatched products receive 'No Color'

Data Quality Validation Checks

Check Type	Columns Checked	Rule
Regex	ProductKey, CustomerKey, OrderDate, Product Name, Brand, Category, Subcategory, Color, City, State, CountryRegion, Name	Must match defined character patterns
Range	ProductKey ≥ 1, CustomerKey ≥ 1, Quantity ≥ 1, Net Price ≥ 0, DateKey ∈ [1,731], Year ∈ [2008,2009]	Numeric bounds enforcement
Enum	Continent (7 values), Education (6 values), Occupation (6 values), Customer_Type (2 values), Color (15 values)	Must be within allowed value set
Null	All key and required columns	Zero nulls allowed
Outlier (IQR)	Quantity, Net Price	Flag values outside 1.5 × IQR fence

2.3 Gold Layer — Dimensional Modeling

The Gold layer builds the final dimensional tables from the cleaned silver DataFrames. Surrogate integer keys are generated for all dimension tables.

Table	Construction Logic
Dim_Brand	Distinct Brand values from silver_sales, sorted A→Z, sequential BrandKey from 1
Dim_Country	Distinct CountryRegion values renamed to Country, sorted, sequential CountryKey
Dim_Customer	One row per CustomerKey using groupby().first() to pick best non-null values. Customer_Type = 'guest' if Education is Unknown, else 'registered'
Dim_Product	One row per ProductKey (deduplicated). BrandKey joined from Dim_Brand. Color extracted from last word of Product Name matched against 14-color palette
Dim_City	Distinct (City, State, Continent, CountryRegion) combinations. CountryKey joined from Dim_Country. Sorted by CountryRegion → State → City
Dim_Date	Continuous date spine from min(OrderDate) to max(OrderDate). Attributes: Year, Quarter, Month, Month Name, Day, Day Name, DateKey
Fact_Sales	Select columns from silver_sales + CityKey joined via (City, State, CountryRegion) lookup. SaleKey = sequential integer row number
Fact_Forecast_2009	silver_forecast joined to Dim_Brand (Brand) and Dim_Country (CountryRegion) to resolve surrogate keys

2.4 Galaxy Schema Validation

After all Gold tables are built, a comprehensive validation pass runs before export:

Validation	Details
Primary Key Uniqueness	Every surrogate key column checked for duplicates and nulls
Foreign Key Integrity	All FK relationships checked; orphan counts reported
Referential Integrity	9 FK relationships verified across Fact and Dim tables
Orphan Detection	Any fact-table FK value not present in parent dimension is flagged

2.5 Output Layer — Export & Round-Trip Verification

- All 8 Gold tables exported to CSV (UTF-8, no BOM) in the Fact__Dimensions/ directory
- Round-trip verification reads each CSV back and confirms row counts match in-memory DataFrames
- BI-ready files produced with consistent surrogate integer keys and no nulls in key columns

3. Data Model — Galaxy Schema

The schema is a Galaxy (Fact Constellation) model containing two fact tables sharing multiple dimension tables. This supports both sales performance analysis and budget-vs-actual comparisons within a single semantic model.

3.1 Schema Diagram (Textual)

3.2 Fact Tables

Fact_Sales (298,246 rows)

Column	Type	Description
SaleKey	int32 PK	Sequential surrogate key (auto-generated)
OrderDateKey	int32 FK	FK → Dim_Date.DateKey
ProductKey	int32 FK	FK → Dim_Product.ProductKey
CityKey	int32 FK	FK → Dim_City.CityKey
CustomerKey	int32 FK	FK → Dim_Customer.CustomerKey
Quantity	int32	Units sold (≥ 1)
Net Price	float64	Net revenue per transaction (≥ 0.0)

Fact_Forecast_2009 (33 rows)

Column	Type	Description
ForecastKey	int32 PK	Sequential surrogate key
ForecastDateKey	int32 FK	FK → Dim_Date.DateKey (always 367 = 2009-01-01)
BrandKey	int32 FK	FK → Dim_Brand.BrandKey
CountryKey	int32 FK	FK → Dim_Country.CountryKey
Forecast	int64	Annual forecasted revenue for Brand × Country combination

3.3 Dimension Tables

Table	Rows	Key Columns & Notes
Dim_Brand	11	BrandKey (PK), Brand — All brands extracted from sales data
Dim_Country	3	CountryKey (PK), Country — 3 countries: China, Germany, United States
Dim_Customer	8,868	CustomerKey (PK), Name, Education, Occupation, Customer_Type
Dim_Product	2,495	ProductKey (PK), Product Name, BrandKey (FK), Category, Subcategory, Color
Dim_City	306	CityKey (PK), City, State, Continent, CountryKey (FK)
Dim_Date	731	DateKey (PK), Date, Year, Quarter, Month, Month Name, Day, Day Name — covers 2008-01-01 → 2009-12-31

3.4 Foreign Key Relationships

Child Table.Column	Parent Table.Column	Cardinality
Fact_Sales.OrderDateKey	Dim_Date.DateKey	Many:1
Fact_Sales.ProductKey	Dim_Product.ProductKey	Many:1
Fact_Sales.CityKey	Dim_City.CityKey	Many:1
Fact_Sales.CustomerKey	Dim_Customer.CustomerKey	Many:1
Fact_Forecast_2009.ForecastDateKey	Dim_Date.DateKey	Many:1
Fact_Forecast_2009.BrandKey	Dim_Brand.BrandKey	Many:1
Fact_Forecast_2009.CountryKey	Dim_Country.CountryKey	Many:1
Dim_Product.BrandKey	Dim_Brand.BrandKey	Many:1
Dim_City.CountryKey	Dim_Country.CountryKey	Many:1

4. Key Assumptions

4.1 DateKey Encoding

DateKey is an integer offset from the anchor date 2008-01-01, where that date = 1. This means DateKey 1 = 2008-01-01, DateKey 366 = 2008-12-31, DateKey 367 = 2009-01-01, and DateKey 731 = 2009-12-31. All fact tables use this same encoding, enabling direct joins to Dim_Date without string-to-date conversion at query time.

4.2 Forecast Granularity

The forecast dataset contains one record per Brand × Country combination for the year 2009 only. All 33 records share ForecastDateKey = 367 (2009-01-01), meaning forecasts are annual-level, not monthly or quarterly. No time-series disaggregation is applied.

4.3 Customer Classification

Customers are classified as 'registered' or 'guest' based on whether Education data is available. If Education = 'Unknown' (meaning no profile data was found in the source), the customer is tagged as 'guest'. This is a derived attribute, not a field present in the source data.

4.4 Product Color Derivation

The Color attribute in Dim_Product is not taken directly from the raw Color field in the source (which was found to be anomalous — often equaling Subcategory). Instead, Color is re-derived by extracting the last word of Product Name and matching it against a fixed 14-color vocabulary: Azure, Black, Blue, Brown, Gold, Green, Grey, Orange, Pink, Purple, Red, Silver, White, Yellow. Products whose last word does not match receive Color = 'No Color'.

4.5 No Row Deletion for Outliers

IQR-based outlier detection is applied to Quantity and Net Price in the Silver layer for observability and logging purposes only. No rows are removed based on outlier status. All transactions proceed to the Gold layer regardless of outlier flags.

4.6 Geography Keys

CityKey is assigned by sorting cities within each country and state and assigning sequential integers. CountryKey follows the same sequential pattern. These keys have no business meaning and should not be compared across pipeline runs if source data changes.

4.7 Dim_Date Spine

The date spine in Dim_Date is generated programmatically to cover every calendar day from the minimum to the maximum OrderDate observed in the sales data (2008-01-01 through 2009-12-31, totaling 731 days). The spine is gapless — no dates are missing within this range.

4.8 Source Data Immutability

The Bronze layer never modifies source files. All transformations occur in-memory on copies of the loaded DataFrames. The pipeline is designed to be fully re-runnable and idempotent: re-execution produces identical output given the same input files.

4.9 Missing Customer Name Handling

Where a customer's Name is null in the source, the pipeline assigns the synthetic name 'Customer_<CustomerKey>' (e.g., 'Customer_42'). This ensures Dim_Customer has no null Name values while preserving traceability to the original key.

4.10 Scope

- Only the data present in Sales.json and forecast.json is processed. No external data enrichment is applied.
- The pipeline is batch-mode only; no streaming or incremental loading is implemented.
- The Power BI file (Sales-Overview.pbix) consumes the CSV outputs from Fact__Dimensions/.
- The three countries in scope are: China, Germany, and United States.

5. Technology Stack & Dependencies

Library	Version	Purpose
Python	3.8+	Core runtime
pandas	Any recent	DataFrame operations, I/O, groupby, merge
pathlib.Path	stdlib	File path management
json	stdlib	JSON deserialization
re	stdlib	Regex-based data quality checks
datetime	stdlib	Execution timing
Power BI Desktop	Latest	BI reporting and dashboarding

5.1 Directory Structure

```
├── data/
│   ├── Sales.json
│   └── forecast.json
├── Fact__Dimensions/
│   ├── Dim_Brand.csv
│   ├── Dim_City.csv
│   ├── Dim_Country.csv
│   ├── Dim_Customer.csv
│   ├── Dim_Date.csv
│   ├── Dim_Product.csv
│   ├── Fact_Sales.csv
│   └── Fact_Forecast_2009.csv
├── ETL_medallion_pipeline.py
├── Sales-Overview.pbix
├── Workflow_Diagram.png
└── Data_Model.png
```